A Motion-Compensated Learned Video Codec with Context-Adaptive Entropy Coding: Measured Bitstreams and a Diagnosis of the Gap to H.264/H.265

***Abstract*—Learned video codecs that beat H.265/HEVC are large, jointly trained systems, so it is hard to tell what each part contributes. We build a small motion-compensated learned video codec and measure every change in real, bit-exactly decodable bytes against libx264, libx265 and two pre-trained learned image codecs, all scored by one harness on the same frames. P-frames use integer-pel block motion compensation and a 13 k-parameter channel-autoregressive entropy model routed to 512 shared frequency tables. Under criteria fixed before results were read, six entropy and quantization changes were admitted and nine rejected. On a held-out DAVIS 2017 split the initial codec needs 5.5× the bits of x264 (BD-rate +450.7%). The measurements place the deficit in the transform and the range-fitted per-channel quantizer. Matching a GDN/residual transform to the baseline's symbol count, then fine-tuning it against quantization noise, cut the all-intra deficit from +176.2% to +87.9% and then +46.5%, as the diagnosis predicted. With the entropy stack refitted and a transmitted scale map, the full codec reaches +210.7% against x264. A factorized-prior image codec still needs about half our all-intra bits, which points to fixed-step quantization trained per rate as the next step.**

Keywords—learned video compression, motion compensation, context-adaptive entropy coding, quantization-aware training, BD-rate, H.264/AVC, H.265/HEVC.

I. Introduction

Hybrid codecs such as H.264/AVC [1] and H.265/HEVC [2] combine hand-designed motion-compensated prediction, block transforms, scalar quantization and context-adaptive arithmetic coding. Learned image codecs replace these tools with networks trained for rate and distortion together [3], and the scale hyperprior [4] and the joint autoregressive and hierarchical prior [5] made them competitive with, and then better than, HEVC intra coding. Learned video codecs extend this across time: DVC [6] reached H.264, scale-space flow [7] simplified motion compensation, FVC [8] moved motion and residual coding into feature space, and the DCVC family [9], [10] passed H.265 and then H.266. These systems use 10–30 M parameters and report the end state of joint training, which leaves open how much of the gain a small codec keeps, and from which parts.

We answer by measurement. We build each part of a small codec, train it on student-scale compute and measure every change in real coded bytes on a held-out split, with libx264, libx265, CompressAI's bmshj2018 models [4], [11] and our codec scored by one harness on the same decoded 8-bit frames. The contributions are: (i) a motion-compensated latent-residual video codec with a bit-exact bitstream from a native C range coder; (ii) a channel-autoregressive residual entropy model made affordable by routing its predictions to 512 shared tables by minimum expected code length; (iii) a gating protocol, with criteria committed before results were read, that admitted six entropy and quantization changes and rejected nine candidates on real coded bytes; (iv) a diagnosis placing the gap in the transform and in range-fitted per-channel quantization, whose prediction about quantization-aware training a later run confirmed; and (v) the full-video result of acting on it, +450.7% to +210.7% against x264, including how much of the learned entropy stack survives a change of transform.

II. Related Work

Ballé et al. [3] set out the analysis–quantization–synthesis structure most learned codecs follow, with generalized divisive normalization (GDN) [12] and additive uniform noise in place of rounding; Theis et al. [13] used a similar differentiable stand-in. Most later gains came from the entropy model: the hyperprior [4] transmits side information that predicts each latent's scale, Minnen et al. [5] added a spatial autoregressive context, and later work used Gaussian mixtures with attention [14] or transformer transforms [15]. Minnen and Singh [16] decode the latent in channel slices, each conditioned on the slices before it, and ELIC [17] makes the groups uneven and adds a spatial context. Our G16 model belongs to this channel-conditional family; it conditions on a motion-compensated reference latent and on the residual's decoded channel groups, with no hyperprior. These codecs round with a fixed step and code the result under a learned density [18]. Our per-channel affine quantizer instead stretches each channel's range over a fixed alphabet, and Section V-E measures what that costs.

Hyperprior implementations in tensorflow-compression and CompressAI [11] map each predicted Gaussian scale onto a fixed table of 64 CDFs. Our codebook does this for categorical predictions: 512 prototypes are fitted to training residuals, and each prediction goes to the prototype of minimum expected code length, (4).

FVC [8] also forms the residual in feature space, but estimates and compensates motion there too; we do both in pixel space. DCVC-FM [19] spans a wide quality range with one model through a learned quantization scale. Coolchic video [20] comes close to AVC with 800 parameters by overfitting a decoder to each video, so small learned video codecs are not new; ours is trained once, and our contribution is a component-by-component account on real bitstreams.

Entropy decoding fails if encoder and decoder compute probabilities that differ in the last bit. Integer networks [21] make decoding consistent across platforms, and DCVC-RT [22] integerizes its model for the same reason; DCVC-RT also finds speed bounded by memory I/O and function-call count more than by arithmetic, as our profiling of a PyTorch block search did (Section IV). Codecs are compared by BD-rate [23] on PSNR and MS-SSIM [24], usually after training on Vimeo-90K [25]; we test on DAVIS [26].

III. Method

Frames are coded in GOPs of ten: an intra-coded first frame, then P-frames predicted from the previous decoded frame. Every symbol, motion vectors included, is entropy coded and counted. Fig. 1 shows the final codec.

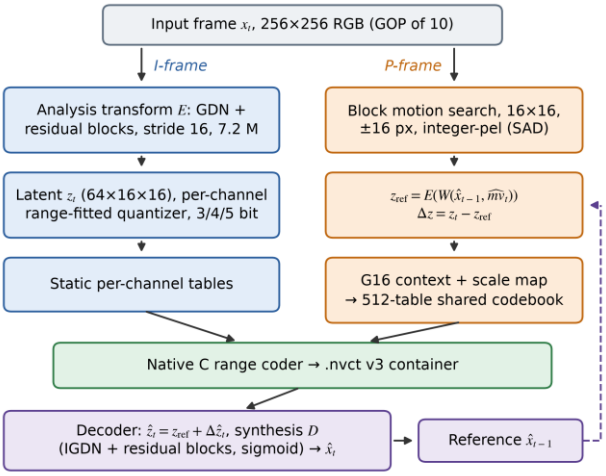


Fig. 1. The final codec. The initial codec differs in its transform (Conv/ReLU, 593 k parameters) and has no scale map.

A. Learned Transforms and Training

The initial transform E is a four-stage convolutional encoder (3→32→64→128→64 channels, each a stride-2 convolution and ReLU) with a mirrored synthesis transform D ending in a sigmoid: 593,411 parameters that map a 256×256 frame to a 64×16×16 latent (16,384 symbols). It was warm-started on Vimeo-90K and fine-tuned on DAVIS TRAIN with Adam (learning rate 10^{-4}, batch 8, 30 epochs, 18,120 steps) and 4-bit per-channel quantization noise under

$$L = d(x, \hat{x}) + \lambda \cdot R(\hat{z}), \quad \lambda = 3\times10^{-4} \tag{1}$$

where d is MSE and R the rate under one learned Laplace density per channel. An earlier R scored against a frozen bin width, and the model learned to shrink its latent 2.5× and cut that proxy 3.7× while real bytes grew, because the deployed quantizer re-fits its grid to the latent's range; making R track the latent's scale removed 97.3% of the effect. Among $\lambda \in \{0, 1, 2, 3, 4.5\}\times10^{-4}$, two seeds each, $\lambda = 3\times10^{-4}$ gave −18.12% BD-rate against the distortion-only control.

The final transform uses GDN/IGDN [12] and residual blocks with 192 internal channels and the same 64-channel latent: 7,208,899 parameters at exactly the baseline's symbol count (Section V-E). It was trained on 256×256 Vimeo-90K crops with horizontal flips, by Adam (10^{-4}, batch 16, gradient-norm clipping at 1.0) for 40 epochs on MSE alone, with the learning rate cut tenfold for the last two of ten data chunks, then fine-tuned for 27 epochs with 4-bit per-channel quantization noise (QAT). Neither stage had a rate term; rate enters only through the quantizer.

B. Motion-Compensated Latent Residual Coding

Warping happens in pixel space, and the residual is formed in latent space because the sigmoid output of D cannot represent signed pixel residuals:

$$x_w = W(\hat{x}_{t-1}, \hat{mv}_t), \quad z_{ref} = E(x_w), \quad \Delta z = z_t - z_{ref} \tag{2}$$

$$\hat{z}_t = z_{ref} + Q^{-1}(Q(\Delta z)), \quad \hat{x}_t = D(\hat{z}_t) \tag{3}$$

Motion is found by full-search block matching (16×16 blocks, ±16 pixels, sum of absolute differences, ties broken toward shorter vectors). Vectors are integer-pel, so the warp is bit-exact, and are coded with two 64-symbol tables; the encoder searches against the decoded reference, so both sides share it. When motion compensation was admitted, vectors took 3.8%, 5.5% and 8.8% of the stream at 5, 4 and 3 bits, saved 2.0× their size in residual bytes at 4 bits against the unwarped previous frame, and were worth −32.22% BD-rate (PSNR) and −41.86% (MS-SSIM) against all-intra coding with that stage's static residual model; all-intra then cost about 1.48× the bits.

C. Context-Adaptive Learned Entropy Coding

Residual symbols dominate P-frames, so the model of Δz was built in three stages.

1) Reference-conditioned model: the decoder holds z_{ref} before any residual symbol, so the residual can be conditioned on it at no side cost. Two 3×3 convolutions (32 channels) and a per-channel embedding predict a categorical distribution at each of the 16,384 positions (11,880–12,672 parameters, trained on rate alone).

2) Channel-autoregressive context: where motion compensation fails, the residual is large in many channels at one position, which z_{ref} does not show but decoded channels do. We add three causal planes per channel (previous-group magnitude, running activity, availability) and decode the 64 channels in four groups of sixteen (G16): 12,744–13,536 parameters, −4.81% BD-rate, with every test sequence improved and the high-motion ones most.

3) Shared codebook: building 16,384 integer frequency tables per frame dominated coding time. Instead, $K = 512$ prototypes q_k are fitted on TRAIN and each prediction p is routed to

$$k^* = \arg\min_k \Sigma_s\, p(s)\cdot(-\log_2 q_k(s)) \tag{4}$$

one matrix product per frame. This keeps the context model's gain to within 0.02% and removes 70–88% of residual-coding time and 91–95% of table memory. Re-fitting the prototypes to the TRAIN histogram with assignments fixed changes no symbol and saved a further 1.42%, 2.64% and 4.04% of bytes at 5, 4 and 3 bits, 99.97–100.02% of the offline prediction.

D. Quantization and Calibration

Latents and residuals use per-channel affine uniform quantizers at $b = 3$, 4 or 5 bits, the three operating points. Channel c's grid spans its 0.1st to 99.9th TRAIN percentile in 2^b levels, so every channel fills the alphabet whatever its energy. Under fixed-step rounding with a learned density [3], [18], a channel within ±0.5 rounds to zero almost everywhere and costs a fraction of a bit per symbol; here it still pays most of the full depth (Section V-E), and the bit depth sets the operating point. Grids and tables are calibrated on 400 TRAIN frames sampled uniformly across sequences (coverage of sequences matters more than frame count: these recover 97.3% of what 576 broadly sampled frames gain over 576 sequential ones). Re-centring the residual grid,

$$h_c = \max(|P_{0.1}(\Delta z_c)|, |P_{99.9}(\Delta z_c)|), \quad \text{grid} = [-h_c, h_c] \tag{5}$$

left the step almost unchanged (ratio 1.02) but moved zero in 37 of 64 channels, cutting 3-bit residual entropy from 1.568 to 1.239 bits per symbol.

E. Gating Protocol

Single changes are worth 1–5%, about what calibration noise or tuning on evaluation data can produce. Every candidate's thresholds and quality guards were therefore committed before results were read. Parameters are fitted on TRAIN; the candidate is tuned on VAL-A (five sequences), confirmed on the disjoint VAL-B (four), verified

in real bytes with bit-exact decoding and provenance hashes, and only then measured once on TEST.

IV. System Implementation

The codec is a Python package with a native C range coder [27], [28] whose output is within 0.026% of the model cross-entropy. The container carries three 8-byte entropy-model identities and per-frame motion and residual lengths (version 3 adds a side-information payload), and a decoder refuses streams whose identities do not match. A codec bundle freezes the transform, grids, context model, codebooks and motion tables and recomputes every identity on load; across six bundles and nine test sequences, 54 of 54 streams were byte-identical to the research path and decoded bit-exactly. Training used one 8 GB laptop GPU and a hosted notebook service.

Profiling the initial encoder on a CPU showed block matching taking 85% of encode time, because the PyTorch version dispatches thousands of small tensor operations per frame, the call-count bound DCVC-RT describes [22]. A C version with AVX2 intrinsics and a scalar fallback, accumulating blocks in the original order so that vectors stay bit-identical, cut encoding from 463 to 96 ms/frame and the motion stage from 437 to 16 ms per P-frame; the two searches agreed on all 720 DAVIS frame pairs. The care is needed: an equally valid floating-point summation order changed 38 of 196,608 codebook assignments, enough to desynchronise encoder and decoder.

V. Results and Discussion

A. Experimental Setup

Data. DAVIS 2017 [26] has 90 train/val sequences at 480p. Frames are resized with area interpolation to a 256-pixel short side and centre-cropped to 256×256, and sequences are split at random (seed 42) into 72 TRAIN, 9 validation and 9 TEST (bmx-bumps, car-turn, cat-girl, cows, drift-chicane, drone, gold-fish, schoolgirls, surf; 719 frames). TEST is our held-out split, not DAVIS test-dev.

Anchors. FFmpeg runs libx264 and libx265 with preset medium at CRF 20–44 (step 2) on rgb24 input, encoding yuv420p and decoding back to RGB, in the default GOP with B-frames and in a low-delay mode matching ours (an I-frame every ten frames, no B-frames, no scene-cut). Intra-only comparisons use x264 all-intra; we omit x265 all-intra because its rate stopped falling at high CRF, a configuration fault we could not resolve. The learned anchors are CompressAI's MSE-trained bmshj2018 factorized and hyperprior models [4], [11] (Vimeo-90K, qualities 1–8), counted in real rANS bytes.

Metrics. Decoded frames are rounded to 8 bits. PSNR (MSE over the three RGB channels, peak 255) and MS-SSIM [24] are computed on RGB, averaged within each sequence and then over sequences; rate is total file size (the whole .mp4 or container) over all pixels. Three other averaging conventions (all frames; MSE pooled per sequence; MSE pooled over all frames) are reported as a check.

BD-rate. Our three operating points are one fewer than the cubic fit of [23] needs, so we interpolate log-rate piecewise-linearly in quality and average the difference over the overlapping range (2,000-point grid); the anchors have 8 or 13 points. Positive means more bits than the anchor, and savings are quoted as percentages of bytes. The overlap is 28.31–29.82 dB for the initial codec and 30.56–32.86 dB for the final one on full video, so the results hold within this harness and these windows.

B. Rate–Distortion Performance

Tables I and II and Fig. 2 give the results; every stream decodes bit-exactly. The initial codec is +450.7% (PSNR) and +378.3% (MS-SSIM) against default x264, about 5.5× the bits. The final codec is +210.7% and +200.6% against x264, +230.1% against x265 and +125.3% against x264 in our GOP structure; at matched quality it needs 3.05×, 3.02× and 3.66× the bits of x264 at 3, 4 and 5 bits.

TABLE I. Operating Points (DAVIS TEST, full video)

Codec / setting	BPP	PSNR (dB)	MS-SSIM
Final codec, 3-bit	0.3546	30.56	0.9661
Final codec, 4-bit	0.5495	32.29	0.9821
Final codec, 5-bit	0.7738	32.86	0.9863
Initial codec, 3-bit	0.3170	28.31	0.9468
Initial codec, 4-bit	0.5043	29.46	0.9676
Initial codec, 5-bit	0.7211	29.82	0.9737
H.264, CRF 24	0.1989	32.63	0.9813
H.264, CRF 28	0.1194	30.67	0.9701
H.264, CRF 32	0.0743	28.77	0.9522
H.265, CRF 24	0.2193	33.25	0.9826
H.265, CRF 28	0.1323	31.29	0.9721

Both curves flatten at the top: from 4 to 5 bits the final codec gains 0.57 dB for 41% more bits (initial codec: 0.36 dB for 43%). One more bit adds about a bit per symbol in every channel, because each channel's range is re-spread over the alphabet, while quality nears the unquantized ceiling (33.09 dB). Learned codecs usually span rates with one model per λ [3]–[5], but with a range-fitted quantizer λ barely moves rate: in the sweep of Section III-A the 4-bit point moved at most 9.7% in rate and 0.68 dB in PSNR, against 41–55% in rate per bit of depth. Per-λ training pays off once quantization uses a fixed step (Section V-F).

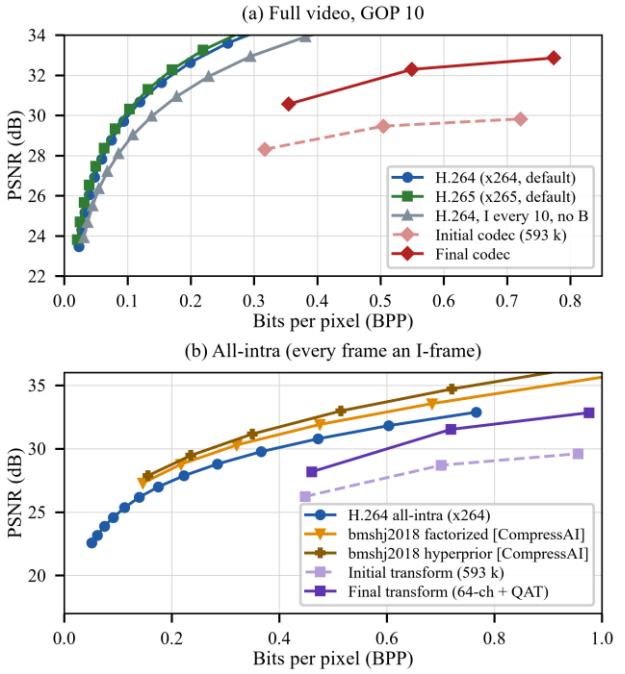


Fig. 2. Rate–distortion curves on DAVIS TEST: (a) full video; (b) all-intra, with the CompressAI learned anchors.

TABLE II. BD-RATE AGAINST EACH ANCHOR (POSITIVE = MORE BITS)

Codec, anchor	PSNR	MS-SSIM
Full video, GOP 10		
Initial, x264 default	+450.7%	+378.3%
Initial, x264 I every 10, no B	+300.5%	+244.7%
Final w/o scale map, x264 default	+214.2%	+203.9%
Final, x264 default	+210.7%	+200.6%
Final, x265 default	+230.1%	+193.4%
Final, x264 I every 10, no B	+125.3%	+117.7%
Final, bmshj2018 factorized (intra)	+7.8%	−1.7%
All-intra, against x264 all-intra		
Initial 593 k transform	+176.2%	+167.6%
64-ch latent, 192-ch body	+87.9%	+110.1%
64-ch, two residual blocks	+101.5%	+118.8%
64-ch + QAT (final)	+46.5%	+55.8%
bmshj2018 factorized	−23.3%	−23.8%
bmshj2018 hyperprior	−31.8%	−27.3%
All-intra, against bmshj2018 factorized		
64-ch + QAT (final)	+90.9%	+99.1%

C. Contribution of Individual Components

Rate-aware training gave −18.12% against its distortion-only control, and motion compensation −32.22% against all-intra. The six admitted entropy and quantization changes were the reference-conditioned model (−1.04% BD-rate over a hand-made lookup), G16 (−4.81%), the codebook (a 0.02% cost for 70–88% less time), prototype re-fitting (1.42–4.04% of bytes), motion-table re-fitting (0.4–0.9%) and residual re-centring (1.06–1.76%, −2.47% BD-rate). Changes confined to entropy coding left PSNR and MS-SSIM bit-identical, and the coder realised 99.97–100.02% of each modelled gain.

D. Negative Results and Diagnosis of the Gap

Nine candidates were rejected: spatial context; a calibration sampling policy; a GOP-boundary motion fix (bounded at 0.36% of the stream); an oracle reference (2.0–14.9%, unrealisable in a decoder); intra-quantizer recalibration; codebook hysteresis; causal reference refinement (0.71% on VAL-B, 0.14% on TEST); and the 192-channel and two-residual-block transforms. None of the first seven touched the transform, and the layers they worked on already run within a few percent of their inputs' entropy, so the work moved to the transform.

Coded all-intra, the initial codec without re-centring is +176.2% against x264 all-intra, and +306.2% when both use an I-frame every ten frames. Forced all-intra at matched PSNR (29.0–29.5 dB), x264 needs 2.8× its bits and ours 1.8×. That is above the 1.48× of Section III-B because the entropy models admitted after motion compensation made only P-frames cheaper. Our motion compensation was the weaker half relative to H.264's, and the transform the larger single deficit.

E. Transform Capacity and Latent Shape

A first GDN/residual transform with a 192-channel latent (8,437,827 parameters) diverged on Vimeo-90K in its third epoch: GDN's gradient is unbounded near its lower bound, and the run had no clipping. Its last finite checkpoint, after two epochs on about a tenth of the data, reaches only 30.56 dB unquantized, below the 32.59 dB of the 64-channel-latent model with the same body. It is undertrained, and we draw no distortion conclusion from it. Its rate result does not depend on training: at equal stride 192 channels code 3× the symbols, and its streams were 3.16–3.26× larger at every bit depth. Rate pressure cannot recover this under our quantizer, which fits each channel's grid to its own spread: a channel with a twentieth of the busiest channel's energy still paid 68% of full depth (Spearman +0.22 between energy and cost). That is a property of range-fitted per-channel quantization; under fixed-step rounding such a channel would cost almost nothing.

We therefore held the symbol count fixed. With 192 internal channels and a 64-channel latent (7,208,899 parameters, 12× the baseline) the transform gates at +87.9% all-intra (+110.1% MS-SSIM), better under all four conventions on both metrics, with streams within 2% of the baseline's at matched bit depth for up to 2.37 dB more PSNR. Two residual blocks (9,864,643 parameters, same symbol count) regressed to +101.5% while fitting Vimeo-90K better (validation loss 1.19× lower), a generalisation gap.

What remains is quantization fragility. Unquantized, the transform leads the baseline by a flat 2.450 dB but loses 0.179, 0.627 and 1.710 dB more to quantization at 5, 4 and 3 bits; the differences, 2.27, 1.82 and 0.74 dB, match the measured 2.37, 1.88 and 0.79 dB, so by 3 bits fragility takes 70% of the gain. Having never seen a quantized latent, the transform should gain most from QAT at the coarsest quantizer. It did: the deficit fell to +46.5% (+55.8% MS-SSIM), better under every convention, at a stream within 0.7% of its previous size. The unquantized ceiling rose from 32.589 to 33.092 dB (0.503 dB from further training), and the quantization loss fell by 0.332, 0.391 and 0.633 dB at 5, 4 and 3 bits. The two sum to within 0.06 dB of the measured 0.86, 0.95 and 1.16 dB, and the fragility share grows from 40% to 56% as the quantizer coarsens, as predicted. A 1.08 dB excess remains at 3 bits.

F. Full-Video Result and Learned Anchors

The transform's full-video effect can be isolated first. Coding both transforms through one simple entropy stack (static per-channel tables, no learned models) gives +528.5% for the initial transform and +248.1% for the final one against x264; the final transform gains 2.22–3.05 dB of PSNR for 6–9% more bytes at matched bit depth. The learned P-frame stack was fitted to the old latent and its provenance check refuses the new one, so each stage was refit to the QAT transform in order. The refit changes only entropy coding (PSNR and MS-SSIM are unchanged) and saves 7.60%, 9.25% and 11.10% of bytes at 5, 4 and 3 bits, giving +214.2% against x264, 236.5 points better than the initial codec. The learned stack keeps about half its value: 33.9 BD-rate points here, against 70.0 on the initial transform (+528.5% to +458.5%), consistent with a transform that decorrelates more leaving less for the entropy model. Residual re-centring is worth 0.05 points here and was dropped.

The residual coder still could not make a symbol's cost depend on content at its position beyond what z_{ref} and decoded channels show. A hand-made scale map addresses part of this: the channel-mean $|\Delta z|$ at each of the 16×16 positions, quantized to 3 bits on TRAIN bins, sent as side information in container version 3 and decoded first to select among joint tables, with no network. On TEST it saved 1.49%, 1.32% and 0.69% of bytes at 5, 4 and 3 bits including its own 62,468 bytes, with PSNR unchanged and exact decoding: +210.7% against x264. On VAL-B a trained hyper-analysis network passed through the same bottleneck carried 2.4–2.9× less.

The learned anchors (Fig. 2b) are −23.3% (factorized) and −31.8% (hyperprior) against x264 all-intra. Our final transform coded all-intra is +90.9% and +115.2% against them, and even the full final codec is +7.8% and +23.7%,

though they code every frame independently. The factorized model is the closest design to ours, a GDN transform at stride 16 trained on Vimeo-90K with no context model or hyperprior; it differs in fixed-step rounding, a learned per-channel density and one model per λ trained for rate. That it needs about half our all-intra bits places the remaining intra gap in the quantizer and the rate objective more than in transform capacity.

G. Per-Frame Coding Behaviour

Fig. 3 shows two P-frames of bmx-bumps. Block search captures the camera pan as near-uniform vectors except on the rider, and the latent residual and the largest errors concentrate on the rider and on fine texture such as the fence. Entropy coding is lossless, so these views of the QAT transform do not depend on it.

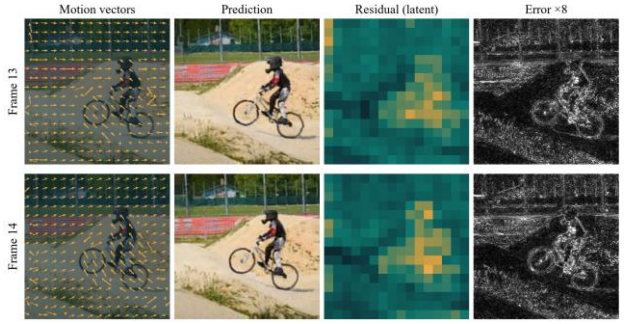


Fig. 3. Two P-frames of bmx-bumps: motion vectors, prediction, latent-residual magnitude and error (×8).

VI. Conclusion

We measured every change to a small motion-compensated learned video codec in real, bit-exact bytes against H.264, H.265 and two learned image codecs. After two large early gains, five of the six admitted entropy and quantization changes saved 0.4–4.8% each, the sixth (the codebook) cut residual-coding time by 70–88%, and nine candidates were rejected. The diagnosis placed the deficit in the transform and the quantizer. Matching the symbol count and then training against the quantizer took the all-intra deficit from +176.2% to +87.9% and +46.5%, the second step as predicted, and a refitted entropy stack with a scale map took the full codec from +450.7% to +210.7% against x264. The halved value of the entropy stack and the factorized-prior anchor both point to the quantizer and rate training as what to change next.

VII. Limitations and Future Work

Results are at 256×256 on DAVIS, not UVG or the HEVC test conditions, so the anchors are measured here instead of taken from published tables. The final transform changes the nonlinearity and adds residual blocks at once, and QAT came with further training, so neither effect is isolated beyond the ceiling decomposition. The 192-channel model was not retrained, so only its rate result stands. Most results use one seed; the λ sweep used two, and at the chosen λ they gave −20.9% and −15.3%. The full-video BD-rates span windows of only 1.5–2.3 dB. Decoding is bit-exact on one machine; cross-platform decoding is untested because codebook assignment depends on the last bit of a floating-point product, which integer networks [21], [22] address. The codec is not real-time: the initial codec encodes at 96 ms/frame and decodes at 118–170 ms/frame on a CPU, and the final transform has 12× its parameters. Next, fixed-step rounding under a learned density, with one model per λ and the scale statistic in the rate term, targets the intra gap the anchors expose; learned optical flow and joint training then target the temporal path, now the larger deficit (+125.3% against x264 in our GOP structure, against +46.5% all-intra).

References

[1] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.

[2] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.

[3] J. Ballé, V. Laparra, and E. P. Simoncelli, "End-to-end optimized image compression," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[4] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," in *Proc. ICLR*, 2018.

[5] D. Minnen, J. Ballé, and G. Toderici, "Joint autoregressive and hierarchical priors for learned image compression," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018.

[6] G. Lu, W. Ouyang, D. Xu, X. Zhang, C. Cai, and Z. Gao, "DVC: An end-to-end deep video compression framework," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 11006–11015.

[7] E. Agustsson, D. Minnen, N. Johnston, J. Ballé, S. J. Hwang, and G. Toderici, "Scale-space flow for end-to-end optimized video compression," in *Proc. IEEE/CVF CVPR*, 2020, pp. 8503–8512.

[8] Z. Hu, G. Lu, and D. Xu, "FVC: A new framework towards deep video compression in feature space," in *Proc. IEEE/CVF CVPR*, 2021, pp. 1502–1511.

[9] J. Li, B. Li, and Y. Lu, "Deep contextual video compression," in *Proc. NeurIPS*, vol. 34, 2021.

[10] J. Li, B. Li, and Y. Lu, "Neural video compression with diverse contexts," in *Proc. IEEE/CVF CVPR*, 2023, pp. 22616–22626.

[11] J. Bégaint, F. Racapé, S. Feltman, and A. Pushparaja, "CompressAI: A PyTorch library and evaluation platform for end-to-end compression research," arXiv:2011.03029, 2020.

[12] J. Ballé, V. Laparra, and E. P. Simoncelli, "Density modeling of images using a generalized normalization transformation," in *Proc. ICLR*, 2016.

[13] L. Theis, W. Shi, A. Cunningham, and F. Huszár, "Lossy image compression with compressive autoencoders," in *Proc. ICLR*, 2017.

[14] Z. Cheng, H. Sun, M. Takeuchi, and J. Katto, "Learned image compression with discretized Gaussian mixture likelihoods and attention modules," in *Proc. IEEE/CVF CVPR*, 2020, pp. 7939–7948.

[15] Y. Zhu, Y. Yang, and T. Cohen, "Transformer-based transform coding," in *Proc. ICLR*, 2022.

[16] D. Minnen and S. Singh, "Channel-wise autoregressive entropy models for learned image compression," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2020, pp. 3339–3343.

[17] D. He, Z. Yang, W. Peng, R. Ma, H. Qin, and Y. Wang, "ELIC: Efficient learned image compression with unevenly grouped space-channel contextual adaptive coding," in *Proc. IEEE/CVF CVPR*, 2022, pp. 5718–5727.

[18] J. Ballé, P. A. Chou, D. Minnen, S. Singh, N. Johnston, E. Agustsson, S. J. Hwang, and G. Toderici, "Nonlinear transform coding," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 2, pp. 339–353, Feb. 2021.

[19] J. Li, B. Li, and Y. Lu, "Neural video compression with feature modulation," in *Proc. IEEE/CVF CVPR*, 2024, pp. 26099–26108.

[20] T. Leguay, T. Ladune, P. Philippe, and O. Déforges, "Cool-chic video: Learned video coding with 800 parameters," in *Proc. Data Compression Conf. (DCC)*, 2024, pp. 23–32.

[21] J. Ballé, N. Johnston, and D. Minnen, "Integer networks for data compression with latent-variable models," in *Proc. ICLR*, 2019.

[22] Z. Jia, B. Li, J. Li, W. Xie, L. Qi, H. Li, and Y. Lu, "Towards practical real-time neural video compression," in *Proc. IEEE/CVF CVPR*, 2025, pp. 12543–12552.

[23] G. Bjøntegaard, "Calculation of average PSNR differences between RD-curves," ITU-T SG16/Q6 VCEG, Doc. VCEG-M33, Austin, TX, USA, Apr. 2001.

[24] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," in *Proc. 37th Asilomar Conf. Signals, Syst. Comput.*, 2003, vol. 2, pp. 1398–1402.

[25] T. Xue, B. Chen, J. Wu, D. Wei, and W. T. Freeman, “Video enhancement with task-oriented flow,” *Int. J. Comput. Vis.*, vol. 127, no. 8, pp. 1106–1125, Aug. 2019.

[26] J. Pont-Tuset, F. Perazzi, S. Caelles, P. Arbeláez, A. Sorkine-Hornung, and L. Van Gool, “The 2017 DAVIS challenge on video object segmentation,” arXiv:1704.00675, 2017.

[27] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Commun. ACM*, vol. 30, no. 6, pp. 520–540, Jun. 1987.

[28] G. N. N. Martin, “Range encoding: An algorithm for removing redundancy from a digitised message,” in *Proc. Video & Data Recording Conf.*, Southampton, U.K., 1979.